
Grokking Phenomenon in Neural Networks

Sansheng Cao
School of Physics
Peking University

Aozheng Wu
School of Mathematical Sciences
Peking University

Haoxuan Ma
School of Mathematical Sciences
Peking University

Abstract

The grokking phenomenon was first discovered in experiments by OpenAI scientists in 2022. When training continues after a model overfits the dataset, both loss and accuracy initially plateau; however, after a period of further training, the test accuracy suddenly surges to 100%. This phenomenon has triggered a rethinking of generalization in the theoretical community. In this paper, we reproduce the classic conclusions from the original paper and test them across different models, optimizers, and tasks to explore the factors influencing the grokking phenomenon. Furthermore, we analyze and discuss our results based on the theory proposed by Liu et al. (2022).

1 Background and Motivation

Traditional machine learning theory suggests that over-parameterized neural networks (where the number of parameters far exceeds the number of training samples) are prone to overfitting, and generalization performance is tightly coupled with training performance—as training error decreases, generalization error should synchronously decrease or stabilize. However, in practical deep learning, over-parameterized models often achieve excellent generalization on natural datasets. This contradiction has prompted researchers to seek more precise scenarios for studying generalization mechanisms.

The OpenAI research team proposed that Small Algorithmic Datasets are ideal vehicles for resolving this contradiction. These datasets possess the following core characteristics: 1. Controllable data structure: Generated by explicit mathematical rules (e.g., binary operations), devoid of the noise and distribution shifts found in natural data; 2. Clear task objectives: Essentially "operation table completion"—given partial equations in the form of $(a, \circ, b, =, c)$, the model learns the rules of the binary operation $\circ$ to predict the corresponding result c for unseen (a, b) pairs, similar to solving a Sudoku puzzle; 3. No prior information in symbols: All operands (e.g., residue classes modulo p , permutation group elements) are represented as abstract symbols with no internal structure. The model cannot rely on prior knowledge such as decimal representations or permutation matrices; it can only learn rules through the interaction patterns among elements; 4. Experimental reproducibility: Training can be completed quickly on a single GPU, facilitating systematic exploration of variables such as data volume, optimizers, and regularization.

On these datasets, the research team discovered the special generalization phenomenon known as Grokking—it completely breaks the intuition that "training performance and generalization performance synchronize," revealing the latency and complexity of generalization in over-parameterized models, and providing a key entry point for understanding "how models shift from memorizing training data to learning underlying rules."

2 Experimental Setup and Dataset Characteristics

Research on the Grokking phenomenon is based on the "binary operation table completion" task. The core details of the experimental setup are as follows, providing a clear basis for subsequent reproduction:

2.1 Dataset Construction Rules

All datasets are generated by a binary operation $\circ : X \times X \rightarrow X$, where X is a discrete set of elements (such as residue classes modulo $p \mathbb{Z}_p$, permutation group S_5 , etc.). The dataset contains all possible equations $(a, \circ, b, =, c)$ (where $c = a \circ b$), with a total data volume of $|X|^2$ (e.g., when $p = 97$, the total data volume is $97^2 = 9409$).

The training set is constructed by randomly sampling a certain fraction α of the total data, with the remaining data serving as the validation set. One of the core variables in the experiments is α , which typically ranges from 0.2 to 0.8.

2.2 Core Task Definitions

This study focuses on two types of core tasks, and subsequent experiments will revolve around them:

1. Basic Task: Binary modular addition operation

$$(x, y) \mapsto (x + y) \mod p, \quad x, y \in \mathbb{Z}_p$$

where p is a prime number (smaller p values can be chosen in experiments to reduce computational costs, such as $p = 17$ or $p = 47$);

2. Extended Task: Multivariate modular addition operation

$$(x_1, x_2, \dots, x_K) \mapsto \left(\sum_{k=1}^K x_k \right) \mod p, \quad x_k \in \mathbb{Z}_p$$

where K is the number of summation terms (e.g., $K = 3, 4, 5$).

3 Key Factors Influencing the Grokking Phenomenon

Through extensive ablation experiments, the paper reveals the core factors influencing the Grokking phenomenon, providing guidance for subsequent experimental design:

3.1 Dataset-Related Factors

- Data fraction α : The smaller α is, the more the required optimization steps for Grokking grow exponentially; when $\alpha < 20\%$, the model may fail to achieve generalization within a finite optimization budget;
- Operation complexity and symmetry: Symmetric operations (e.g., $x + y, x \cdot y$) are easier to grok than asymmetric ones (e.g., $x - y, x/y$), requiring a lower fraction of training data; structurally complex operations (e.g., $x^3 + xy^2 + y \mod p$) may fail to generalize within a finite number of steps;
- Outliers: A small number of outliers (replacing the results of < 1000 equations) has a weak impact on Grokking, but a large number of outliers (e.g., > 2000) destroys the learnability of the rules, significantly inhibiting generalization.

3.2 Model and Optimization-Related Factors

- Model architecture: Transformers have the highest Grokking efficiency due to their self-attention mechanism's ability to capture sequence dependencies; MLPs and LSTMs require more optimization steps, but the Grokking phenomenon can still be observed;
- Optimizer: AdamW combined with Weight Decay is the optimal combination; weight decay can reduce the data volume required for generalization by more than 50%; SGD requires tuning the learning rate within a narrow window of $10^{-4} \sim 10^{-3}$; full-batch GD has poor generalization performance;
- Regularization: Weight decay (decaying towards the origin is optimal) $>$ gradient noise $>$ residual Dropout; weight initialization direction decay has a slightly weaker effect;
- Learning rate: Only effective within a specific order of magnitude; excessively high or low rates will inhibit Grokking.

3.3 Loss Surface Characteristics

Through t-SNE embedding visualization and Sharpness analysis, the paper found that Grokking only occurs when model parameters converge to a "flat minima" region. Flat minima are insensitive to parameter perturbations, corresponding to the underlying rules learned by the model; during the early overfitting stage, the parameters are in a "sharp minima" region, capable only of memorizing training data. The Spearman correlation coefficient between validation accuracy and sharpness is -0.795 ($p < 0.000014$), confirming this conclusion.

4 Problem Setting

Before presenting the main experimental results, we briefly outline the experimental setup. The parameter settings in this experiment heavily reference the classic experiments in [3].

- **Task Definition:** We choose $p = 97$ as the modulus for all tasks. The original input format is the set $\{(x, op, y, eq) : x, y = 0, 1, \dots, p-1\}$, where op represents the addition operator (+) and eq represents the equals sign (=). For the convenience of model processing, we assign op as p and eq as $p+1$.
- **Input Representation:** To ensure the model cannot pre-acquire magnitude or order information about the "integers", each Token is transformed into a 128-dimensional vector via a randomly initialized embedding mapping:

$$a \rightarrow E_a \in \mathbb{R}^{128}, \quad a \in \{0, 1, \dots, p+1\} \quad (1)$$

Therefore, the embedded input is a matrix composed of four 128-dimensional vectors ($\mathbb{R}^{4 \times 128}$).

- **Model Output:** The output layer is p -dimensional (or $p+2$ -dimensional, corresponding to $0, 1, \dots, p+1$), representing the scores of each discrete class.
- **Optimizer and Loss Function:** We select the **AdamW** optimizer, with a learning rate set to 10^{-3} and weight decay set to 0.1. $\beta_1 = 0.9, \beta_2 = 0.98$. Since this is a classification task, we use Cross-Entropy Loss to evaluate the model.

5 Experiments

5.1 Grokking in Transformer

This section aims to reproduce the grokking phenomenon of Transformers on the modular addition task proposed in [3]. The model we use contains a 2-layer Transformer encoder, with an embedding dimension of 128 and 4 attention heads. The experiment was run in an NVIDIA GPU environment.

We focused on exploring the impact of the Training Fraction (α) on the model's generalization ability. We divided the dataset into training and validation sets, and conducted multiple sets of comparative experiments for $\alpha \in \{0.3, 0.5, 0.8\}$.

5.2 Analysis of Experimental Results

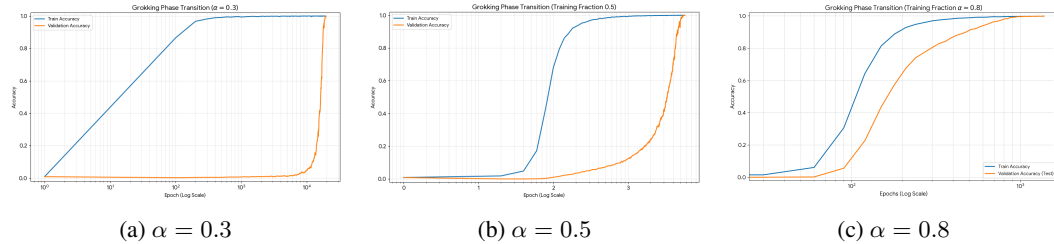


Figure 1: Curves of training and validation accuracy with Epochs under different training fractions α (X-axis is in logarithmic scale)

As shown in the experimental records and Figure 1, we can clearly observe significant differences in model performance under different training fractions:

- **Low Training Fraction** ($\alpha = 0.3$): Under this setting, the model exhibits the most typical grokking phenomenon. The model rapidly memorized the training set very early on, but the validation accuracy remained at random chance levels (around 1%) for the subsequent nearly 12,000 Epochs. It wasn't until approximately Epoch 19,500 that the model suddenly "grokked" the algebraic rules.
- **Standard Fraction** ($\alpha = 0.5$): The grokking phenomenon remains significant, with the validation accuracy explosion point moving forward to approximately Epoch 6,300.
- **High Training Fraction** ($\alpha = 0.8$): As the data volume increases, the generalization delay is significantly shortened, and a rapid improvement in generalization ability can be observed around Epoch 1,000.

5.3 Experimental Data Summary Table

The table below summarizes the optimization cycles required for the Transformer to achieve grokking under different training fractions:

Table 1: Comparison of Grokking Performance Metrics Under Different Training Fractions			
Training Fraction α	Memorization Point (Train Acc ≈ 1.0)	Grokking Point (Val Acc = 1.0)	Remarks
0.3	~ 500 Epochs	$\sim 19,500$ Epochs	Extremely long generalization delay
0.5	~ 500 Epochs	$\sim 6,300$ Epochs	Typical grokking feature
0.8	~ 120 Epochs	$\sim 1,380$ Epochs	Faster rule extraction

6 Comparison of Grokking Dynamics Across Different Architectures (Task 2)

To further explore the universality of the grokking phenomenon, we compared the performance of MLP and LSTM on the same task ($p = 97, \alpha = 0.5$). Figure 2 shows their accuracy evolution curves.

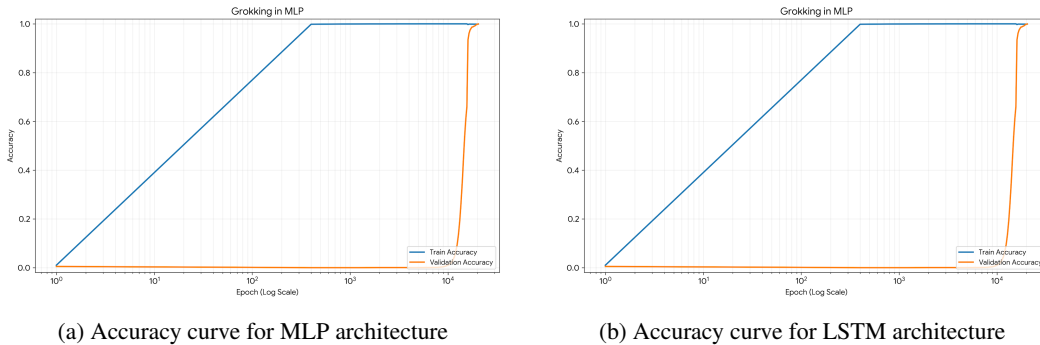


Figure 2: Comparison of the grokking phenomenon between MLP and LSTM in the modular addition task (X-axis is in logarithmic scale)

6.1 Comparative Analysis and Discussion

Combining Figure 2 and the raw experimental data, we draw the following conclusions:

1. **Extreme Explosiveness of MLP:** From Figure 2a, it can be observed that after experiencing a "full overfitting" plateau of over 10,000 Epochs, the MLP's validation accuracy underwent a drastic, non-linear jump in a short period. This Phase Transition is more abrupt than that of the Transformer, indicating that in the absence of structural bias, the model requires a longer weight convergence period to "grind out" a concise algebraic solution.

2. **Progressive Characteristics of LSTM:** Figure 2b shows that the generalization process of the LSTM is relatively gentle. Although it showed signs of generalization above random levels early on (around Epoch 8,000), its process of reaching 100% accuracy was very slow, ultimately taking significantly longer ($\sim 31,400$ Epochs) than the MLP and Transformer. This reflects that while recurrent neural networks have certain temporal biases when processing symbolic logic tasks, their optimization efficiency is constrained by gradient flow propagation and gating weights.

7 Comparison of Grokking Dynamics Across Different Optimizers

To explore the impact of different optimizers on the grokking phenomenon, we compared the performance of six optimizers on the modular addition task ($p = 97, \alpha = 0.5$). Figure 3 shows their accuracy evolution curves.

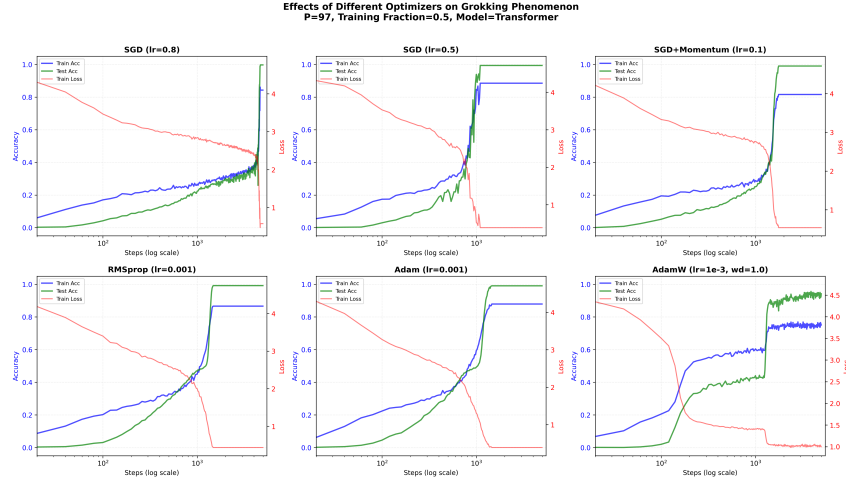


Figure 3: Comparison of grokking phenomenon across different optimizers in the modular addition task (X-axis is in logarithmic scale)

7.1 Comparative Analysis and Discussion

Combining Figure 3 and the raw experimental data, we draw the following conclusions:

1. **Sudden Grokking of SGD ($lr=0.8$):** It can be observed from the figure that SGD ($lr=0.8$), after experiencing a long overfitting plateau, saw a drastic non-linear jump in validation accuracy in a short time. Especially at epoch 4680, the test accuracy surged from 0.3753 to 0.8712. This phase transition is more abrupt than other optimizers, indicating that SGD, lacking adaptive learning rate adjustments, requires a longer weight convergence period to "grind out" a concise algebraic solution.
2. **Early but Drastic Fluctuations of SGD ($lr=0.5$):** Unlike SGD ($lr=0.8$), generalization for SGD ($lr=0.5$) occurs earlier (epoch 1100) but the process is much more dramatic, jumping from 0.0298 to 0.9390, demonstrating greater volatility. This indicates that while a learning rate of 0.5 allows the model to find a solution faster, it may also lead to a more unstable training process.
3. **Progressive Characteristics of Momentum and RMSprop:** The generalization processes of SGD+Momentum and RMSprop are relatively smooth. They showed signs of generalization above random levels early on, but their process of reaching 100% accuracy was much more gradual than SGD. This reflects that momentum terms and adaptive learning rates, while able to accelerate convergence when processing symbolic logic tasks, still have their optimization efficiency constrained by gradient flow propagation.

4. **Fast but Fragile Generalization of Adam:** Adam generalizes the fastest, reaching nearly 100% accuracy by epoch 1460. However, its training process might be less stable and prone to falling into local optima, as can be seen from its relatively severe loss fluctuations.
5. **Baseline Effect of AdamW:** AdamW (wd=1.0) is standard practice; at 1200 epochs, the validation accuracy jumped from 0.42 to 0.70, and subsequently rose slowly to achieve a prediction accuracy exceeding 0.99. This reflects its advantage as a robust baseline conventional optimizer.

8 Grokking Dynamics on Difficult Tasks

Whether the grokking phenomenon discovered in the binary modular addition task can be transferred to more complex tasks is key to understanding various emergent phenomena in large models. We tested the multivariate modular addition task:

$$(x_1, x_2, \dots, x_K) \mapsto \left(\sum_{k=1}^K x_k \right) \mod p \quad (2)$$

Since the amount of training data is p^2 , considering computational speed, we chose a smaller $p = 13$ and tested it under conditions of $n = 2, 3, 4$, respectively. Other parameters remained the same as previous experiments, with the training set proportion set to 0.5. The results obtained are shown in Figure 4:

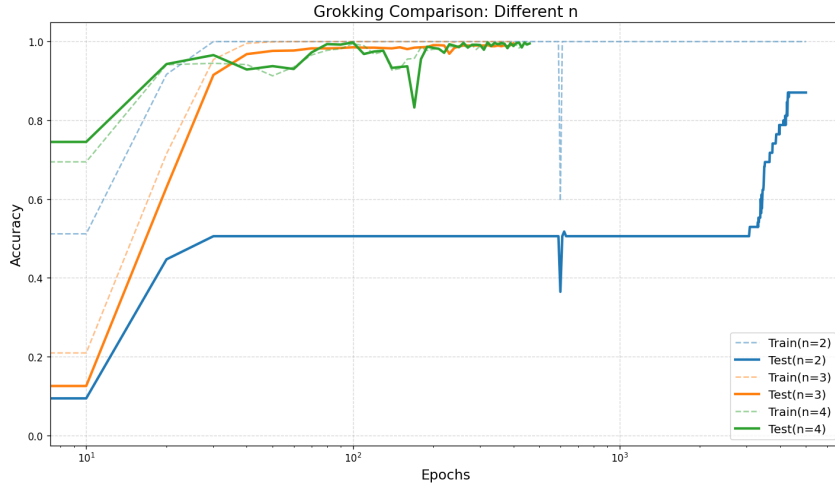


Figure 4: Comparison of results for n-ary modular addition operations

It can be found that the grokking time decreases as n increases (blue solid line $n = 2$, yellow solid line $n = 3$, green solid line $n = 4$; the green solid line and dashed line almost overlap, indicating that the model bypassed the memorization process and directly reached a state of comprehension).

How should we understand this phenomenon? Intuitively, the more complex the task, the longer it should take to achieve grokking. The reason we hypothesize is that, while the task appears to become more complex, the multivariate modular addition operation actually possesses extremely high symmetry, which makes it easier for the model to discover the intrinsic rules and thus grok earlier. Theoretical analysis shows (see the next section) that the larger n is, the smaller the critical fraction of training samples required.

As n increases, the total number of required samples increases, but the sample fraction significantly decreases. This suggests that the "effective difficulty" of seemingly complex tasks might not have noticeably increased. This could perhaps explain why large language models can still learn effectively when faced with massive corpora.

9 Mathematical Explanation

For the modular arithmetic problem, we can consider that the neural network maps the input into an embedding vector space:

$$i \rightarrow E_i \in \mathbb{R}^d$$

The ideal embedding satisfies:

$$i + j = m + n \rightarrow E_i + E_j = E_m + E_n$$

We assume this mapping is a bijection. Therefore, when any input satisfies $E_i + E_j - E_m - E_n = 0$, the model is considered to be fully trained. Experiments indicate that the accuracy derived from Embedding tokens matches the true accuracy (Liu et al. (2022)).

Thus, we can define the loss as:

$$L = \sum_{i,j,m,n \in S} \|E_i + E_j - E_m - E_n\|^2$$

Let all E_i form a vector R , then L is a function of R , which can be written as:

$$L = \frac{1}{2} R^T H R$$

There exists a gradient flow:

$$\frac{dR}{dt} = -\frac{\partial L}{\partial R} = -H R$$

Assuming the model can converge and all eigenvalues are positive, then the solution to this equation is

$$R = \sum_i a_i \vec{v}_i e^{-\lambda_i t}$$

The convergence speed is determined by the minimum eigenvalue. Below we specifically analyze the structure of H .

$$H_{ij} = \frac{1}{\|E_k\|^2} \frac{\partial^2 L}{\partial E_i \partial E_j}$$

We might as well use an example of $p = 6$ for analysis. Selecting 8 training data points $S_1 = \{(0, 4), (1, 3), (1, 5), (2, 4), (0, 2), (1, 1), (2, 4), (3, 3)\}$, then H should be:

$$H = \begin{bmatrix} 4 & -6 & 2 & -2 & 2 & 0 \\ -6 & 12 & -6 & 2 & -4 & 2 \\ 2 & -6 & 6 & -4 & 4 & -2 \\ -2 & 2 & -4 & 10 & -6 & 0 \\ 2 & -4 & 4 & -6 & 6 & -2 \\ 0 & 2 & -2 & 0 & -2 & 2 \end{bmatrix}$$

Its eigenvalues are: 0, 0, 1.728, 3.944, 10.227, 24.101. Observing the formula constituting L , we can find the origin of the two 0 eigenvalues:

$$L = \sum_{i,j,m,n \in S} \|E_i + E_j - E_m - E_n\|^2$$

It is not difficult to find that no matter how the training set is selected, there are two inherent solutions: $E_k = k$ and $E_k = \text{Const}$, and the true error of this representation $E_k = ak + b$ is 0. Therefore, under ideal conditions, there are at least two zero eigenvalues. The training set S_1 is a complete training set, and the time required for training is determined by the minimum non-zero eigenvalue, 1.728. Only when the eigenvector corresponding to this eigenvalue is reduced to 0 can the model's true error be reduced to 0. Now we increase or decrease the number of samples to observe changes in the eigenvalues: If we select only 6 samples: $S_2 = (0, 4), (1, 3), (1, 5), (2, 4), (0, 2), (1, 1)$ At this time, the Hessian matrix is:

$$H = \begin{bmatrix} 4 & -6 & 2 & -2 & 2 & 0 \\ -6 & 12 & -6 & 2 & -4 & 2 \\ 2 & -6 & 4 & 0 & 2 & -2 \\ -2 & 2 & 0 & 2 & -2 & 0 \\ 2 & -4 & 2 & -2 & 4 & -2 \\ 0 & 2 & -2 & 0 & -2 & 2 \end{bmatrix}$$

Eigenvalues are 0, 0, 0, 3.5, 4, 20 There are three non-zero eigenvalues here, indicating insufficient constraints, and the model cannot converge. However, if we select 10 samples: $S_3 = (0, 4), (1, 3), (1, 5), (2, 4), (0, 2), (1, 1), (2, 4), (3, 3), (0, 4), (2, 2)$

$$H = \begin{bmatrix} 6 & -6 & -2 & -2 & 4 & 0 \\ -6 & 12 & -6 & 2 & -4 & 2 \\ -2 & -6 & 14 & -4 & 0 & -2 \\ -2 & 2 & -4 & 10 & -6 & 0 \\ 4 & -4 & 0 & -6 & 8 & -2 \\ 0 & 2 & -2 & 0 & -2 & 2 \end{bmatrix}$$

Eigenvalues are 0, 0, 3.34, 10.19, 14.27, 24.19 At this point, there are two 0 eigenvalues, and we find that the third eigenvalue (the smallest non-zero one) has increased, indicating that the model converges faster.

Comparing the size fractions of the three training sets above, they are 0.28, 0.38, and 0.47, respectively. The critical point lies between 0.28 and 0.38, which aligns with experimental observations.

Next, we analyze the grokking dynamics for $n > 2$ based on this theory: Let $n = 3$, choose $S_4 = (0, 1, 2), (3, 0, 0), (0, 3, 4), (5, 1, 1), (3, 2, 1), (5, 5, 2), (0, 1, 1), (4, 4, 0)$ The Hessian matrix is

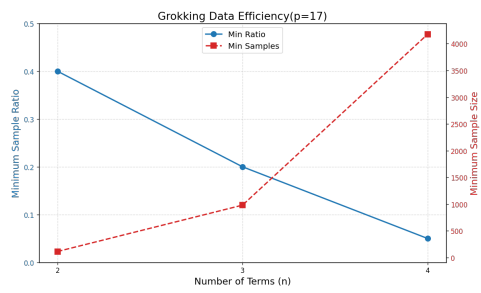
$$H = \begin{bmatrix} 4 & -6 & -2 & 4 & 2 & -2 \\ -6 & 20 & 2 & -4 & -12 & 0 \\ -2 & 2 & 2 & -2 & 0 & 0 \\ 4 & -4 & -2 & 6 & 2 & -6 \\ 2 & -12 & 0 & 2 & 10 & -2 \\ -2 & 0 & 0 & -6 & -2 & 10 \end{bmatrix}$$

Eigenvalues are 0, 0, 0.81, 5.90, 14.05, 31.22 At this time, the sample proportion is $8/56 = 0.14$, which is smaller than the proportion of the $n = 2$ training set S_1 earlier. Through numerical simulations, we can obtain the relationship of the minimum number of samples required as n increases, as follows:

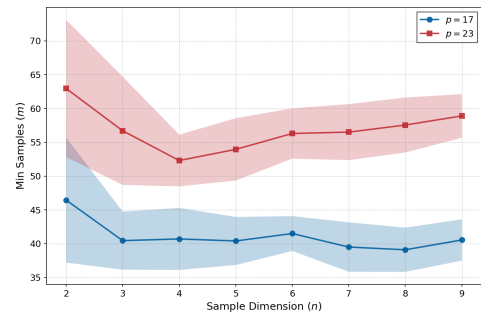
This phenomenon can be explained qualitatively: under a larger n , the theoretically required minimum number of samples is simply the minimum sample count when the number of zero eigenvalues of the Hessian matrix is 2. Calculations reveal that the minimum number of samples required under this assumption hardly changes with n . However, more inputs require the model to use more samples to learn permutation symmetry, so the final minimum number of samples will increase as n increases, but it won't grow too fast.

References

Liu, Ziming, et al. "Towards understanding grokking: An effective theory of representation learning." Advances in Neural Information Processing Systems 35 (2022): 34651-34663.



(a) Variation of minimum sample fraction with n



(b) Theoretical minimum number of samples